

Contents

1	README	1
2	toc	4
3	01-concepts	5
4	02-first-login	6
5	03-tui-navigation	8
6	04-roles-and-permissions	10
7	05-logs-and-debugging	11
8	06-users-and-configs	13
9	07-lobbies-and-rooms	15
10	08-client-signals	17
11	09-maintenance-mode	19
12	10-releases-and-cdn	20
13	11-scheduler	22
14	12-bots-discord-twitch	23
15	13-cleanup-and-backups	25
16	14-incident-playbooks	26
17	15-command-reference	28
18	16-training-moderators	31
19	17-admin-runbook	33
20	glossary	34
21	README	35
22	emergency-change-policy	36
23	platform-sync-policy	37

1 README

SekaiLink Core Access

SekaiLink Core Access est le cockpit d'administration de SekaiLink. Il est pense comme un outil TUI Rust lance depuis le bastion Nexus apres une connexion SSH, puis un login SekaiLink/Nexus.

Le nom Core Access evite la confusion avec Client Core et les libretro cores de Sekaiemu.

Objectifs

- Donner aux Admins et au role Service un environnement de travail unique.

- Centraliser logs, lobbies, rooms, releases, packs CDN, bots, scheduler, maintenance, notes et audit.
- Garder Nexus comme source d'identite, de droits, d'audit et de documentation operationnelle.
- Former les nouveaux modérateurs avec des guides imprimables et une reference de commandes complete.

Table des matieres

- [Concepts](01-concepts.md)
- [Premier login](02-first-login.md)
- [Navigation TUI](03-tui-navigation.md)
- [Roles et permissions](04-roles-and-permissions.md)
- [Logs et debugging](05-logs-and-debugging.md)
- [Users et configs](06-users-and-configs.md)
- [Lobbies et rooms](07-lobbies-and-rooms.md)
- [Signaux clients](08-client-signals.md)
- [Maintenance mode](09-maintenance-mode.md)
- [Releases et CDN](10-releases-and-cdn.md)
- [Scheduler](11-scheduler.md)
- [Bots Discord/Twitch](12-bots-discord-twitch.md)
- [Cleanup et backups](13-cleanup-and-backups.md)
- [Playbooks incidents](14-incident-playbooks.md)
- [Reference commandes](15-command-reference.md)
- [Formation modérateurs](16-training-moderators.md)
- [Runbook Admin](17-admin-runbook.md)
- [Glossaire](glossary.md)
- [Change control](change-control/README.md)

Regles fortes

- Aucun secret reel dans les docs, PDF ou exemples.
- Toute modification future doit etre documentee dans le meme commit que le code.
- Aucune modification SKLMI sans accord explicite ecrit de Jade.
- Les changements de connectivite Client Core, Sekaiemu ou SKLMI exigent un contrat documente.
- Les actions dangereuses demandent une cible tapee, une raison et un audit.

Generation PDF

Les PDF imprimables sont produits dans `dist/pdf/`:

- `sekailink-core-access-full.pdf`
- `sekailink-core-access-service-training.pdf`
- `sekailink-core-access-command-reference.pdf`
- `sekailink-core-access-incident-playbooks.pdf`
- `sekailink-core-access-quick-reference.pdf`

Commande prevue:

```
```sh
bash docs/sekailink-core-access/scripts/build-pdf.sh
```
```

La pipeline utilise Pandoc avec XeLaTeX. Le script signale clairement les dependances manquantes.

Executable

Le binaire local vit dans `tools/core-access/`. Il lance par default un cockpit terminal plein ecran avec panels, hotkeys, ligne de commande, autocompletion, historique, notes, approvals locales, audit JSONL et dashboard local.

Le shell ligne-par-ligne historique reste disponible avec `--shell`.

Commandes de base:

```
```sh
cargo run --manifest-path tools/core-access/Cargo.toml
cargo run --manifest-path tools/core-access/Cargo.toml -- --shell
cargo run --manifest-path tools/core-access/Cargo.toml -- --command "server status all"
```
```

2 toc

Table des matieres imprimable

1. Concepts
2. Premier login
3. Navigation TUI
4. Roles et permissions
5. Logs et debugging
6. Users et configs
7. Lobbies et rooms
8. Signaux clients
9. Maintenance mode
10. Releases et CDN
11. Scheduler
12. Bots Discord/Twitch
13. Cleanup et backups
14. Playbooks incidents
15. Reference commandes
16. Formation modérateurs
17. Runbook Admin
18. Glossaire
19. Change control

Raccourcis globaux

| | |
|--------|-----------------------|
| Touche | Fonction |
| --- | --- |
| F1 | Help |
| F2 | Command palette |
| F3 | Search |
| F4 | Note depuis selection |
| F5 | Refresh |
| F6 | Filter |
| F7 | Export |
| F8 | Tail logs |
| F9 | Actions |
| F10 | Quit |
| F11 | Layout/workspace |
| F12 | Panic |

3 01-concepts

01 - Concepts

Core Access

Core Access est le cockpit d'exploitation de SekaiLink. Il ne remplace pas les services; il les orchestre avec des droits, des confirmations et un audit centralise.

Bastion Nexus

L'outil est lance sur Nexus apres SSH. Le modele d'accès est volontairement en deux etapes:

1. utilisateur Linux autorise sur le bastion;
2. utilisateur SekaiLink/Nexus avec role Admin ou Service.

Un mapping explicite associe le compte Linux au `user\_id` SekaiLink attendu.

Services

- Nexus: identite, utilisateurs, configs, audit, scheduler, approvals.
- Link: API publique, lobbies, chat, room admin passthrough, client signals.
- Worlds: generation et logs de generation.
- Evolution: distribution, CDN, packs, releases.
- Pulse: assistant/config helper et health.

Ops commit

Chaque action mutante produit un ops commit dans Nexus. Un ops commit est une unite atomique d'audit: commande, acteur, cible, raison, diff, logs lies, resultat, erreur et rollback.

Approval queue

Le role Service peut preparer certaines actions sensibles. L'action reste en attente jusqu'a approbation Admin. L'approbation ou le refus est lui-meme audite.

Maintenance mode

Maintenance mode est un etat central qui informe le client et bloque certaines actions selon le scope: full, generation_only, cdn_only, read_only, lobby_only ou client_update_only.

Realtime client signals

Core Access peut envoyer des signaux aux clients SekaiLink: broadcast, maintenance banner, force update prompt, refresh lobby/room, relogin, reconnect SKLMI, diagnostics request.

Les diagnostics demandent le consentement de l'utilisateur.

4 02-first-login

02 - Premier login

Preconditions

- Avoir un compte Linux autorise sur le bastion Nexus.
- Avoir un compte SekaiLink avec role Admin ou Service.
- Avoir un mapping Linux user -> SekaiLink user_id dans Nexus.

Connexion

```
```sh
ssh nexus-vps
sekailink-core-access
```
```

Core Access affiche ensuite un prompt de login SekaiLink.

Verification apres login

Executer:

```
```text
auth whoami
```
```

Resultat attendu:

- Linux user courant.
- SekaiLink user_id.
- Role.
- Capabilities.
- Heure d'expiration de session.

Premier controle

Executer:

```
```text
server status all
release current
maintenance status
```
```

Si un serveur est en rouge, ouvrir ses logs avec F8 ou:

```
```text
server logs <server> <service> --follow
```
```

SSH explicite

Core Access garde l'ouverture SSH interactive derriere une confirmation et une gate locale:

```
```text
ssh open <server> <reason> --confirm ssh:<server>:open
ssh open <server> <reason> --confirm ssh:<server>:open --execute
```
```

Pour executer vraiment, l'environnement local doit contenir:

```
```sh
export SEKAILINK_CORE_ACCESS_SSH_OPEN=1
```
```

Chaque tentative cree un draft audite local avant l'ouverture.

Updates serveurs

Les commandes update ne lancent pas de package manager et ne redemarrent aucun service. Elles servent a creer un plan audite et une trace d'intention:

```
```text
server update plan <server> <scope>
server update apply <server> <plan_id|reason> --confirm server-update:<server>:apply
```
```

Avant une application réelle hors Core Access, faire au minimum:

- `ops snapshot <label>`;
- `server agent-health <server> --execute`;
- `server agent-services <server> --execute`;
- backup des données pertinentes;
- maintenance/broadcast si le scope est visible utilisateur.

5 03-tui-navigation

03 - Navigation TUI

Layout par défaut

```
```text
SekaiLink Core Access
Nexus OK | Link WARN | Worlds OK | Evolution OK | Pulse DOWN
Matrix Live filtered logs Alerts/Jobs
SERVER CPU RAM ERR 22:41 room 38293 joined approvals: 2
Link 38 64 12 22:42 SKLMI reconnect pack check: 14m
... ...
```

```
skl> room logs pas-pour-certo --follow
```

```
```
```

Command line

La ligne de commande est disponible partout.

Fonctions presentes dans le cockpit MVP:

- autocomplete par prefixe avec Tab;
- fleches gauche/droite;
- correction au milieu de ligne;
- historique avec fleche haut/bas;
- Ctrl+A/E;
- Ctrl+W;

Fonctions planifiees:

- selection de suggestion avancee;
- selection visuelle de plage dans les panels logs;
- Alt+B/F;
- reverse search;
- edition multi-line JSON;
- execution non bloquante de flux logs live.

Hotkeys MVP

```
```text
F1 Help
F2 Server status
F3 Command registry
F4 Prepare incident note
F5 Refresh view
F6 Log catalog
F7 Audit search
F8 Prepare log tail
F9 Drop to shell mode
F10 Clear output
F11 Cycle workspace
F12 Panic view
```
```

`Ctrl+Q` ou `Ctrl+C` quitte le cockpit. Le mode `--shell` garde l'ancien prompt ligne-par-ligne pour les sessions de logs longues.

Tabs, splits et workspaces

- F11 change ou sauvegarde le layout.

- Un workspace debug capture les panels ouverts, filtres, pins et notes.
- Les workspaces sont lies a un incident, lobby, room, user ou release.
- Le MVP implemente les workspaces incident via `incident open/status/note/pin/export/close`.

Selection de logs

MVP actuel:

1. Chercher avec `logs search <query> [source|all]`.
2. Isoler avec `logs filter user:<id> lobby:<id> source:<source|all>`.
3. Epingler avec `logs pin <source> <text>`.
4. Ajouter une note avec `logs note <source> <text>`.
5. Exporter avec `logs export [query] --format md`.

Selection visuelle planifiee:

1. Placer le curseur sur le panel log.
2. Selectionner une plage.
3. F4 pour creer une note depuis la selection.
4. F7 pour exporter la selection.

6 04-roles-and-permissions

04 - Roles et permissions

Admin

Admin a le controle complet, avec confirmations et audit:

- reboot/update services;
- publish/rollback release;
- modifier users et roles;
- modifier configs utilisateur;
- give item;
- maintenance full;
- secrets set/rotate;
- approval queue.

Service

Service est quasi-admin live, mais les actions dangereuses demandent approval.

Service peut:

- lire logs, users, configs, lobbies, rooms;
- moderer lobby/chat;
- demander diagnostics client;
- request SKLMI reconnect;
- preparer un give item;
- preparer un rollback;
- preparer une fermeture de room/lobby sensible;
- poster annonces Discord/Twitch selon policy.

Service ne peut pas directement:

- reboot/update serveur;
- publier/rollback release;
- delete user;
- modifier roles;
- modifier secrets;
- modifier DB critique;
- give item sans approval.

Confirmation forte

Toute action dangereuse exige:

- cible tapee exactement;
- raison;
- resume d'impact;
- ops commit.

Exemple:

```
```text
Type target to confirm: link
Reason: emergency restart after stuck release-latest deploy
```
```

7 05-logs-and-debugging

05 - Logs et debugging

Sources

Core Access doit lister deux vues:

- par serveur/service;
- par incident/correlation.

Sources prioritaires:

- journalctl systemd;
- fichiers logs app;
- nginx/CDN;
- Nexus auth/admin;
- Link chat/lobby;
- Worlds generation;
- Evolution packs/releases;
- Pulse;
- Discord social bot;
- Twitch assistant;
- room multiserver logs;
- room events;
- DB/backup logs;
- client reports.

Commandes de base

```
```text
logs list
logs list --by-server
logs list --by-incident
logs tail <source>
logs search <query> [source|all]
logs filter user:<id> lobby:<id> room:<id> correlation:<id> source:<source|all>
logs pin <source> <text>
logs note <source> <text>
logs export [query] --format md
```
```

Etat MVP actuel:

- `logs search` et `logs filter` produisent un plan dry-run par défaut;
- `--execute` reste bloqué sans `SEKAILINK\_CORE\_ACCESS\_REMOTE\_READONLY=1`;
- les prefixes `user:`, `lobby:`, `room:`, `correlation:`, `item:` et `runtime:` sont normalisés en valeurs de recherche;
- `logs pin` écrit une preuve locale dans `log-pins/pins.jsonl`;
- `logs note` écrit une note locale ciblée `log:<source>`;
- `logs export` produit un fichier local Markdown, JSONL ou TXT sous `exports/`;
- les logs client Sekaiemu/SKLMI doivent passer par une demande `client diagnostics-request` avec consentement utilisateur;
- aucune ligne de log n'est écrite dans Nexus DB dans cette tranche.

Notes liées

Une note liée capture:

- extrait original;
- source;
- timestamps;

- tags;
- cible;
- ops_commit_id;
- auteur.

Verbose filtre

Le cockpit doit etre verbose, mais filtre par default:

- erreurs et warnings visibles en haut;
- details disponibles en drilldown;
- bruit groupable par service, room, user ou correlation.

Admin-Agent

Les commandes `server agent-\*` utilisent les admin-agents loopback via SSH:

- `server agent-health [server|all]` prepare `GET /health`;
- `server agent-system [server|all]` prepare `GET /system`;
- `server agent-services [server|all]` prepare `GET /services`;
- `server agent-service <server> <service>` prepare `GET /services/{service}`;
- `server agent-logs <server> <service>` prepare `GET /services/{service}/logs`.

Execution live:

- `--execute` reste bloqué sans `SEKAILINK\_CORE\_ACCESS\_REMOTE\_READONLY=1`;
- les routes protegee demandent le token admin-agent du serveur;
- les tokens sont lus depuis l'environnement local et ne sont jamais imprimes;
- Pulse n'a pas encore de profil admin-agent dans Core Access; utiliser `health probe pulse` en attendant un agent Pulse declare.

Variables attendues:

- `SEKAILINK\_CORE\_ACCESS\_NEXUS\_AGENT\_ADMIN\_TOKEN`
- `SEKAILINK\_CORE\_ACCESS\_LINK\_AGENT\_ADMIN\_TOKEN`
- `SEKAILINK\_CORE\_ACCESS\_WORLDS\_AGENT\_ADMIN\_TOKEN`
- `SEKAILINK\_CORE\_ACCESS\_EVOLUTION\_AGENT\_ADMIN\_TOKEN`
- `SEKAILINK\_CORE\_ACCESS\_AGENT\_ADMIN\_TOKEN` comme fallback generique hors Nexus

Controle service:

- `server restart|start|stop` utilise `POST /services/{service}/{action}`;
- role Admin requis;
- `SEKAILINK\_CORE\_ACCESS\_REMOTE\_MUTATION=1` requis;
- confirmation exacte `--confirm <server>:<service>:<action>` requise.

8 06-users-and-configs

06 - Users et configs

Users

Commandes principales:

```
```text
user search <query>
user open <user>
user create <user> <email> <role> --password-env ENV --confirm user:<user>:create
user edit <user> key=value --confirm user:<user>:edit
user disable <user> --confirm user:<user>:disable
user sessions <user>
user devices <user>
user audit <user>
user revoke-sessions <user> --confirm user:<user>:revoke-sessions
user force-password-reset <user> --confirm user:<user>:force-password-reset
```
```

Etat MVP actuel:

- `user search`, `user open`, `user sessions`, `user devices` et `user audit` sont connectees au Nexus Identity admin HTTP en read-only;
- l'execution live exige `--execute`,
`SEKAILINK\_CORE\_ACCESS\_REMOTE\_READONLY=1` et
`SEKAILINK\_CORE\_ACCESS\_NEXUS\_IDENTITY\_ADMIN\_TOKEN`;
- `SEKAILINK\_CORE\_ACCESS\_NEXUS\_ADMIN\_TOKEN` reste un fallback local compatible;
- `user create`, `user edit`, `user disable`, `user revoke-sessions` et
`user force-password-reset` utilisent les routes HTTP admin Nexus Identity confirmees;
- toute mutation exige le role Admin, `--execute`,
`SEKAILINK\_CORE\_ACCESS\_NEXUS\_MUTATION=1`, le token Identity admin, et la confirmation exacte affichee dans le dry-run;
- `user create` lit le mot de passe depuis `--password-env ENV` pour eviter de l'inscrire dans l'historique shell ou dans l'audit local; le body affiche en dry-run remplace toujours le mot de passe par ``.

Note importante: `user open`, `user sessions`, `user devices` et `user audit` declenchent une entree d'audit admin cote Identity, meme si elles ne modifient pas le compte cible.

Les mutations utilisateur declenchent egalement l'audit admin cote Nexus Identity. Core Access garde une trace locale de la commande et de son statut, mais ne journalise jamais la valeur des tokens ni le mot de passe env.

Champs `user edit` supportes:

```
```text
email=<email>
display_name=<name>
avatar_url=<url>
bio=<text>
locale=<locale>
role=<player|service|moderator|admin>
email_verified=<true|false>
disabled=<true|false>
permissions=<permission-a,permission-b>
```
```

Ces commandes ne modifient pas SKLMI, Sekaiemu, Client Core, les packs, ni les serveurs room. Elles appellent uniquement le contrat Nexus Identity admin deja

present cote serveur.

Configs utilisateur

Nexus reste la source de verite. Les YAML generes ne sont pas edites comme source.

Commandes:

```
```text
user configs <user_id> [game_key] [--execute]
user config open <user_id> <config_id> [--execute]
user config diff <user_id> <config_id> <version>
user config export <user_id> <config_id> --format yaml [--execute]
user config edit <user_id> <config_id> key=value [key=value...] --confirm user-config:<user_id>:<config_id>:edit
```
```

Etat actuel:

- `user configs` utilise `GET /users/{user\_id}/seed-configs` sur le Nexus seed-config API prive;
- `user config open` utilise la meme route avec `config\_id` comme filtre operateur, car la route detail par config n'est pas encore confirmee;
- `user config export` utilise `POST /users/{user\_id}/seed-configs/{config\_id}/export-yaml`;
- `user config diff` ecrit un draft audite local tant que la route de diff par version n'est pas confirmee;
- `user config edit` ecrit un draft audite local et ne modifie pas les valeurs canoniques Nexus;
- les routes seed-config attendent un `user\_id` Nexus numerique; utiliser `user open <username>` pour resoudre un username avant d'interroger les configs;
- l'execution live exige `--execute`, `SEKAILINK\_CORE\_ACCESS\_REMOTE\_READONLY=1` et `SEKAILINK\_CORE\_ACCESS\_NEXUS\_SEED\_CONFIG\_ADMIN\_TOKEN` (`SEKAILINK\_CORE\_ACCESS\_NEXUS\_ADMIN\_TOKEN` reste fallback).

Regles d'edition

- Admin peut preparer une modification avec diff et ops commit; l'application live du patch attend une route serveur confirmee.
- Service peut lire/diff/export.
- Service peut preparer une edition via approval.
- Chaque edition documente impact Client Core, generation, room et rollback.

9 07-lobbies-and-rooms

07 - Lobbies et rooms

Lobbies

Commandes:

```
```text
lobby list
lobby open <lobby>
lobby create <lobby_id> <name> [visibility] [owner_username] [description] --confirm lobby:<lobby_id>:create
lobby edit <lobby_id> key=value [key=value...] --confirm lobby:<lobby_id>:edit
lobby close <lobby_id> --confirm lobby:<lobby_id>:close
lobby lock <lobby_id> [reason] --confirm lobby:<lobby_id>:lock
lobby unlock <lobby_id> [reason] --confirm lobby:<lobby_id>:unlock
lobby chat <lobby_id>
lobby join-secret <lobby_id> [reason] --confirm lobby:<lobby_id>:join-secret
lobby broadcast <lobby_id> <message> --confirm lobby:<lobby_id>:broadcast
```
```

`lobby join-secret` est invisible pour les joueurs ordinaires, mais jamais pour l'audit. Il doit apparaitre comme `admin\_observer`.

Etat MVP actuel:

- `lobby list [limit] [query] [visibility] [status] [offset]` prepare un GET prive vers Nexus lobby-admin;
- `lobby open <lobby>` prepare un GET prive vers Nexus lobby-admin;
- `lobby create`, `lobby edit` et `lobby close` utilisent les routes privees Nexus lobby-admin confirmees, avec `--execute`, `SEKAILINK\_CORE\_ACCESS\_NEXUS\_MUTATION=1` et confirmation exacte;
- `lobby lock`, `lobby unlock`, `lobby broadcast` et `lobby join-secret` creent des drafts audites locaux;
- `lobby chat` prepare une recherche de logs sur `link:chat-api`;
- l'execution live est read-only, bloquee par default, et exige `--execute`, `SEKAILINK\_CORE\_ACCESS\_REMOTE\_READONLY=1` et `SEKAILINK\_CORE\_ACCESS\_NEXUS\_LOBBY\_ADMIN\_TOKEN`;
- `SEKAILINK\_CORE\_ACCESS\_NEXUS\_ADMIN\_TOKEN` reste un fallback local compatible;
- les mutations lobby live sont bloquees sans role/gate/confirmation.

Rooms

Commandes:

```
```text
room list [limit] [query] [room_type] [connection_state] [offset]
room open <room_id>
room summary <room_id>
room events <room_id> [limit] [event_type] [severity] [offset] [source]
room logs <room_id> --follow
room snapshot <room_id>
room sync <room_id> [reason] --confirm room:<room_id>:sync
room pending-items <room_id>
room client-reports <room_id> [limit] [report_type] [severity] [source] [offset]
room request-sklmi-reconnect <room_id> <player> [reason] --confirm room:<room_id>:request-sklmi-reconnect
room disconnect-runtime <room_id> <player|runtime> [reason] --confirm room:<room_id>:disconnect-runtime
room give-item <room_id> <target> <item> [reason] --confirm room:<room_id>:give-item
```
```

Etat actuel:

- `room list` utilise `GET /rooms` sur Nexus room-query;

- `room open` et `room snapshot` utilisent `GET /rooms/{room\_id}`;
- `room summary` utilise `GET /rooms/{room\_id}/diagnostics`;
- `room events` utilise `GET /rooms/{room\_id}/events`;
- `room client-reports` utilise `GET /rooms/{room\_id}/client-reports`;
- `room pending-items` lit le snapshot et demande d'inspecter
`pending\_delivery\_count`/`received\_items`;
- `room logs` prepare une recherche `link:room-runtime` ou un follow
`sekailink-room-server`;
- `room sync`, `room request-sklmi-reconnect`, `room disconnect-runtime` et
`room give-item` creent des drafts audites locaux, sans mutation runtime.

Execution read-only room-query:

- `--execute`
- `SEKAILINK\_CORE\_ACCESS\_REMOTE\_READONLY=1`
- `SEKAILINK\_CORE\_ACCESS\_NEXUS\_ROOM\_QUERY\_ADMIN\_TOKEN`
- `SEKAILINK\_CORE\_ACCESS\_NEXUS\_ADMIN\_TOKEN` fallback

Give item

Policy:

- Admin ou approval Admin.
- Snapshot automatique avant action.
- Raison obligatoire.
- Impact live affiche.
- Ops commit.

SKLMI reconnect

`room request-sklmi-reconnect` envoie un signal client/runtime. Une modification SKLMI pour supporter une nouvelle forme de reconnect demande accord explicite.

10 08-client-signals

08 - Signaux clients

Les signaux clients sont realtime. Le fallback poll peut exister pour robustesse, mais le cockpit doit viser l'immediat.

Signaux

```
```text
client broadcast
client maintenance-banner
client force-update-prompt
client request-relogin
client refresh-lobby
client refresh-room
client request-sklmi-reconnect
client restart-runtime
client clear-cache-request
client diagnostics-request
client diagnostics-list
client diagnostics-export
```
```

Diagnostics

Regles:

- consentement utilisateur obligatoire;
- logs et configs redacted;
- upload lie a un incident ou ops commit;
- jamais de secret client brut.

Etat Core Access actuel:

- `client diagnostics-request` cree une demande locale `draft-consent-required`;
- `client broadcast`, `client maintenance-banner`, `client force-update-prompt`, `client request-relogin`, `client refresh-lobby`, `client refresh-room`, `client request-sklmi-reconnect`, `client restart-runtime` et `client clear-cache-request` creent des drafts locaux audites dans `drafts/client-signal.jsonl`;
- les signaux sensibles exigent `--confirm client:<action>:<target>`;
- `client refresh-lobby` et `client refresh-room` sont des drafts sans confirmation forte, car ils ne sont pas destructifs;
- `client clear-cache-request` reste marque approval-required dans le detail du draft;
- aucun signal client n'est envoye par Core Access actuellement;
- aucun fichier Client Core, Sekaiemu ou SKLMI n'est collecte par Core Access;
- `client diagnostics-list` liste les demandes locales;
- `client diagnostics-export` exporte les demandes et le contrat de bundle attendu.

Bundle attendu quand l'integration client sera implementee:

- `client-core/main.log`, `client-core/renderers.log`, `client-core/updater.log`;
- `sekaiemu/sekaiemu.log`, `sekaiemu/stdout.log`, `sekaiemu/stderr.log`;
- `sklmi/sklmi.log`, `sklmi/stdout.log`, `sklmi/stderr.log`;
- configs redacted et contexte de lancement;
- informations systeme sans secrets.

Toute modification SKLMI necessaire pour produire ces fichiers demande accord explicite ecrit de Jade avant implementation.

Broadcast

Ciblage:

- global;
- server;
- lobby;
- room;
- role;
- version;
- game.

Exemple:

```
```text
broadcast lobby pas-pour-certo "Maintenance de room dans 5 minutes." --confirm broadcast:lobby:pas-pour-certo
```
```

Etat actuel: les commandes `broadcast ...` creent des drafts locaux audites dans `drafts/broadcast.jsonl`. Aucun message n'est envoye aux clients, rooms, lobbies, Discord ou Twitch tant que le transport serveur n'est pas connecte.

11 09-maintenance-mode

09 - Maintenance mode

Scopes

- `full`
- `generation\_only`
- `cdn\_only`
- `read\_only`
- `lobby\_only`
- `client\_update\_only`

Etat

Champs:

- enabled;
- scope;
- message;
- starts_at;
- ends_at;
- allow_admin_bypass;
- created_by;
- reason;
- ops_commit_id.

Commandes

```
```text
maintenance status
maintenance enable <scope> <message> --confirm maintenance:<scope>:enable
maintenance disable [scope] [reason] --confirm maintenance:<scope>:disable
maintenance schedule <scope> <start> <end> <message>
maintenance broadcast <scope> <message> --confirm maintenance:<scope>:broadcast
maintenance history
```
```

Etat Core Access actuel:

- `maintenance schedule` cree un draft local dans `maintenance/current.txt` et `maintenance/history.jsonl`;
- `maintenance enable`, `maintenance disable` et `maintenance broadcast` creent des drafts locaux audites dans `drafts/maintenance.jsonl`;
- aucune commande maintenance ne modifie encore le mode maintenance serveur ou client;
- les actions sensibles exigent une confirmation exacte.

F12 Panic

F12 ouvre un panneau controle:

- maintenance full;
- freeze generation;
- broadcast incident;
- snapshot active rooms;
- collect logs bundle.

Chaque action Panic demande confirmation forte.

12 10-releases-and-cdn

10 - Releases et CDN

Releases

Commandes:

```
```text
release current
release list
release verify [latest|version|date|manifest] [--fast]
release verify-cdn [channel] [platform|all] [--execute]
release compare <a>
release publish <manifest|version|date> --confirm release:<version>:publish
release rollback <version> --confirm release:<version>:rollback
release schedule <manifest|version|date> <datetime> --confirm release:<version>:schedule
release notes [version|date|manifest]
release audit [version|date|manifest]
```
```

Etat Core Access actuel:

- `release current` lit le dernier manifeste local sous `apps/client-core/release/update-bundles/YYYYMMDD`;
- `release list` deduplique les manifestes `client\_release\_manifest.fragment.json` et `sekailink-client-release-YYYYMMDD.json`;
- `release verify` verifie existence, taille et SHA-256 des bundles locaux;
- `release verify --fast` saute le SHA-256 pour une verification rapide;
- `release verify-cdn` affiche les probes publics `release-latest`; `--execute` appelle seulement l'API publique, sans token;
- `release compare` compare version, channel, build, base URL, artefacts, tailles et hash;
- `release publish`, `release rollback` et `release schedule` creent des drafts locaux audites, avec confirmation exacte. Ils ne modifient pas le CDN, ne remplacent pas `/opt/sekailink/link/chat-api/config/client\_release\_latest.json` et ne redemarrent pas `sekailink-chat-api.service`.

Validation

Une release valide montre:

- version;
- platform;
- update bundle URL;
- fallback URL;
- SHA;
- taille;
- date upload;
- manifest backup disponible.

Trois bannieres Client Core

Trois slots fixes en haut du dashboard Client Core:

```
```text
client-banner list
client-banner edit <1|2|3>
client-banner preview <1|2|3>
client-banner publish <1|2|3>
client-banner rollback <1|2|3>
client-banner disable <1|2|3>
```
```

Champs: title, body, type, action label, action URL/deeplink, color/accent, icon, enabled, starts_at, ends_at, priority.

Packs CDN

Pipeline obligatoire:

1. check;
2. validate;
3. stage;
4. publish.

Commandes:

```
```text
pack repo list
pack repo add <id> <url> <game> [notes] --confirm pack-repo:<id>:add
pack repo edit <id> key=value [key=value...] --confirm pack-repo:<id>:edit
pack repo disable <id> [reason] --confirm pack-repo:<id>:disable
pack repo delete <id> [reason] --confirm pack-repo:<id>:delete
pack check <id> [notes]
pack validate <id> [notes]
pack stage <id> [notes] --confirm pack:<id>:stage
pack publish <id> [notes] --confirm pack:<id>:publish
pack rollback <id> <version> --confirm pack:<id>:rollback:<version>
pack schedule-check <id> <cron|interval> --confirm pack:<id>:schedule-check
```
```

Etat actuel:

- les commandes `pack repo add/edit/disable/delete` creent des drafts locaux audites;
- `pack check` et `pack validate` creent des drafts Service sans fetch reseau;
- `pack stage`, `pack publish`, `pack rollback` et `pack schedule-check` creent des drafts Admin avec confirmation exacte;
- aucun pack n'est telecharge, aucun artifact CDN n'est stage/publie, aucune logique Lua n'est executee, et aucun comportement SKLMI/Sekaitemu n'est modifie par cette tranche.

13 11-scheduler

11 - Scheduler

Types de jobs

- release client;
- pack check;
- pack publish;
- maintenance window;
- backup;
- health probe;
- Discord announcement;
- Twitch announcement;
- cleanup;
- release verification.

Commandes

```
```text
schedule list
schedule calendar
schedule add <name> <when> <command> --confirm schedule:<name>:add
schedule edit <job> key=value [key=value...] --confirm schedule:<job>:edit
schedule pause <job> [reason] --confirm schedule:<job>:pause
schedule resume <job> [reason] --confirm schedule:<job>:resume
schedule run-now <job> [reason] --confirm schedule:<job>:run-now
schedule history
```
```

Etat Core Access actuel:

- `schedule add` ajoute un draft local dans `scheduler/jobs.jsonl`;
- `schedule edit`, `schedule pause`, `schedule resume` et `schedule run-now`
creent des drafts locaux audites dans `drafts/schedule.jsonl`;
- aucun job n'est arme, modifie ou execute automatiquement;
- toutes les actions Admin exigent une confirmation exacte.

Champs job

- owner;
- type;
- target;
- interval ou cron;
- next_run;
- last_run;
- last_success;
- last_error;
- enabled;
- approval_required;
- ops_commit_id.

Calendrier

Le calendrier doit etre lisible par jour, semaine et mois. Les jobs sensibles affichent leur approval status.

14 12-bots-discord-twitch

12 - Bots Discord et Twitch

Les configs bots vivent dans Nexus et sont auditees. Les secrets sont set/rotate seulement, jamais affiches en clair. Core Access ne lit pas les fichiers `XX-API\*.local.md` et ne montre aucun token.

La premiere integration est volontairement defensive:

- `status` et `logs` preparent des lectures via les services allowlistes.
- `--execute` reste bloqué par `SEKAILINK\_CORE\_ACCESS\_REMOTE\_READONLY=1`.
- Les annonces, reloads, commandes, timers et connexions creent des drafts audites dans `~/sekailink/core-access/drafts/`.
- Aucune API Discord/Twitch n'est appelee par ces commandes.

Discord

Commandes:

```
```text
discord status [--execute]
discord logs [--follow] [--execute]
discord reload --confirm discord:reload:bot
discord announce <channel> <message> --confirm discord:announce:<channel>
discord sync-roles --confirm discord:sync-roles:all
discord command list
discord command enable <name> [detail] --confirm discord:command:<name>:enable
discord command disable <name> [detail] --confirm discord:command:<name>:disable
discord command edit <name> key=value [key=value...] --confirm discord:command:<name>:edit
discord timer list
discord timer edit <id> key=value [key=value...] --confirm discord:timer:<id>:edit
discord incident-post <incident> [channel] [message] --confirm discord:incident-post:<incident>
```
```

`discord status` pointe vers le service `sekailink-social-bots` sur Link.

`discord logs` pointe vers la source `discord:bot`.

Twitch

Commandes:

```
```text
twitch status [--execute]
twitch logs [--follow] [--execute]
twitch connect <channel> [reason] --confirm twitch:connect:<channel>
twitch disconnect <channel> [reason] --confirm twitch:disconnect:<channel>
twitch announce <channel> <message> --confirm twitch:announce:<channel>
twitch command list
twitch command enable <name> [detail] --confirm twitch:command:<name>:enable
twitch command disable <name> [detail] --confirm twitch:command:<name>:disable
twitch command edit <name> key=value [key=value...] --confirm twitch:command:<name>:edit
twitch timer list
twitch timer edit <id> key=value [key=value...] --confirm twitch:timer:<id>:edit
twitch lobby announce <lobby> [message] --confirm twitch:lobby:<lobby>:announce
twitch stream set-title-hint [channel] <title>
```
```

`twitch status` pointe vers le service `sekailink-twitch-assistant` sur Link.

`twitch logs` pointe vers la source `twitch:assistant`.

Drafts

Les drafts bot sont append-only et peuvent etre relus avec:

```
```text
discord command list
discord timer list
twitch command list
twitch timer list
ops timeline discord
ops timeline twitch
```
```

Chaque draft inclut `id`, `ts`, `state`, `author`, `domain`, `action`, `target` et `detail`. Pour appliquer plus tard ces drafts vers Nexus ou vers les bots live, il faudra ajouter une integration serveur documentee avec gate d'environnement et confirmations exactes.

Rollback

Toute modification de commande, timer ou mapping doit etre versionnee et rollbackable.

15 13-cleanup-and-backups

13 - Cleanup et backups

Scope

Cleanup v1 couvre:

- logs;
- caches;
- fichiers temporaires;
- old update bundles;
- generation spool;
- sessions expirees;
- vieux lobbies;
- anciens audits archives;
- stale room artifacts.

Commandes

```
```text
cleanup plan logs
cleanup plan db
cleanup plan spool
cleanup plan all
cleanup apply <plan_id> [reason] --confirm cleanup:<plan_id>:apply
cleanup history
cleanup rollback <id> [reason] --confirm cleanup:<id>:rollback
```
```

Etat Core Access actuel:

- `cleanup plan ...` cree un draft local audite dans `drafts/cleanup.jsonl`;
- `cleanup apply` et `cleanup rollback` creent des drafts Admin avec confirmation exacte;
- aucun scan, aucune suppression, aucun backup, aucune mutation DB et aucun changement serveur n'est execute par Core Access dans cette tranche.

Regles

- Dry-run obligatoire.
- Estimation fichiers/lignes/bytes/rows.
- Backup gate pour DB cleanup.
- Confirmation cible + raison.
- Ops commit avec rapport detaille.

Backup gate

Avant une action dangereuse, Core Access verifie le dernier backup. Si le backup est absent ou trop vieux, l'action est bloquee ou demande une approval Admin explicite selon la policy.

16 14-incident-playbooks

14 - Playbooks incidents

Room server down

```
1. `incident open room-<room> sev2 room server symptoms`
2. `room list`
3. `room summary <room>`
4. `room logs <room> --follow`
5. `server status link`
6. `logs filter room:<room> source:link:room-runtime`
7. `incident pin room-<room> link:room-runtime <important line>`
8. `incident note room-<room> <decision or next step>`
9. Si necessaire: `ops snapshot room-<room>`.
```

SKLMI non connecte

```
1. `incident open sklmi-<room>-<player> sev3 SKLMI not connected`
2. `room summary <room>`
3. Verifier presence player et runtime.
4. `room events <room>`
5. `logs filter runtime:<player> source:link:room-runtime`
6. `incident pin sklmi-<room>-<player> link:room-runtime <important line>`
7. `client request-sklmi-reconnect <player>`
8. Si echec: `client diagnostics-request <player> sklmi-<room>-<player> <reason> --include sekaiemu,sklmi,client`
9. Exporter la demande si besoin avec `client diagnostics-export <player> --file diagnostics-<player>.md`.
```

Client update brise

```
1. `release current`
2. `release verify-cdn`
3. `logs filter release:<version>`
4. `maintenance enable client_update_only --message "...`
5. `release rollback <version>` si Admin.
```

Generation bloquee

```
1. `server status worlds`
2. `logs tail worlds:generation`
3. `schedule history health-probe-worlds`
4. `maintenance enable generation_only --message "...`
```

CDN pack invalid

```
1. `pack repo list`
2. `pack validate <id>`
3. `pack rollback <id> <version>`
4. `broadcast game <game> "...`
```

Bot Discord/Twitch down

```
1. `discord status` ou `twitch status`
2. `discord logs` ou `twitch logs`
3. `discord reload` ou `twitch connect <channel>`
4. Creer note incident si recurring.
```

Maintenance d'urgence

```
1. F12 Panic.
2. Lire impact.
3. `incident open emergency-<scope> sev1 <summary>`
4. Choisir scope minimal.
```

5. Taper cible + raison.
6. Broadcast.
7. Collect logs bundle.
8. ``incident export emergency-<scope> --file emergency-<scope>.md``

Releve apres incident

1. ``ops timeline <label>``
2. ``incident status <label>``
3. ``incident export <label> --file <label>.md``
4. ``ops handoff shift-<date> --file shift-<date>.md``

``incident export`` isole un incident. ``ops handoff`` resume le shift complet: incidents, notes, pins, approvals, maintenance, scheduler, packs, banners et audit local.

17 15-command-reference

15 - Reference commandes

Chaque commande indique le role minimal. Les actions marquees Approval peuvent etre preparees par Service mais executees seulement apres approbation Admin.

| Commande | Role | Confirmation | Description |
|---|---------|--------------|--|
| --- | --- | --- | --- |
| `auth whoami` | Service | Non | Affiche l'identite Linux/SekaiLink et capabilities. |
| `dashboard` | Service | Non | Affiche la matrice locale Core Access MVP. |
| `commands list` | Service | Non | Liste le registre de commandes connu du binaire. |
| `commands search <query>` | Service | Non | Recherche dans le registre de commandes. |
| `server status <server all> --execute` | Service | Non | Statut CPU/RAM/disk/uptime/services; live read-only si gate e |
| `server services <server>` | Service | Non | Liste services systemd connus. |
| `server agent-health [server all] --execute` | Service | Non | Lit `/health` des admin-agents loopback via SSH; gate r |
| `server agent-system [server all] --execute` | Service | Non | Lit `/system` des admin-agents avec token serveur et ga |
| `server agent-services [server all] --execute` | Service | Non | Lit `/services` des admin-agents avec token serveur e |
| `server agent-service <server> <service> --execute` | Service | Non | Lit etat/state d'un service declare dans l'admin |
| `server agent-logs <server> <service> --execute` | Service | Non | Lit le tail JSON expose par l'admin-agent pour un s |
| `nexus services --execute` | Service | Non | Liste les services connus par le private Nexus admin-agent; dry-run par d |
| `server logs <server> <service> --follow --execute` | Service | Non | Suit les logs d'un service; execution distante b |
| `server restart <server> <service> --confirm <server>:<service>:restart --execute` | Admin | Oui | Redemarre un servic |
| `server start <server> <service> --confirm <server>:<service>:start --execute` | Admin | Oui | Demarre un service decl |
| `server stop <server> <service> --confirm <server>:<service>:stop --execute` | Admin | Oui | Arrete un service declare |
| `server update plan <server> [scope reason]` | Admin | Non | Cree un draft/checklist d'update serveur; aucune mutation |
| `server update apply <server> <plan_id reason> --confirm server-update:<server>:apply` | Admin | Oui | Cree un draft d |
| `ssh open <server> [reason] --confirm ssh:<server>:open --execute` | Admin | Oui | Ouvre une session SSH explicite seu |
| `health probe <server all> --execute` | Service | Non | Lance/verifie une sonde health; dry-run par default. |
| `logs list` | Service | Non | Liste les sources logs. |
| `logs list --by-server` | Service | Non | Vue logs par serveur/service. |
| `logs list --by-incident` | Service | Non | Vue logs correlees incident. |
| `logs tail <source> --execute` | Service | Non | Suit une source log; dry-run par default. |
| `logs search <query> [source all] --execute` | Service | Non | Recherche read-only dans les logs allowlistes; dry-run |
| `logs filter <term...> source:<source all> --execute` | Service | Non | Filtre read-only par user/lobby/room/correlati |
| `logs pin <source> <text>` | Service | Non | Epingle une ligne ou un extrait comme preuve locale. |
| `logs note <source> <text>` | Service | Non | Cree une note locale ciblee `log:<source>`. |
| `logs export [query] --format <md jsonl txt> --file <name>` | Service | Non | Exporte pins et notes logs locaux. |
| `audit export [query] [file-name]` | Service | Non | Exporte des lignes d'audit locales dans le dossier exports Core A |
| `note add <target> <text>` | Service | Non | Ajoute une note locale liee a un incident, log, user, lobby ou room. |
| `note list [query]` | Service | Non | Liste les notes locales, optionnellement filtrees. |
| `note export [query] [file-name]` | Service | Non | Exporte des notes locales dans le dossier exports Core Access. |
| `approval request <command> <reason>` | Service | Non | Demande une approbation Admin pour une commande sensible. |
| `approval list` | Service | Non | Liste les demandes et decisions locales. |
| `approval approve <id> [reason]` | Admin | Oui | Approuve localement une demande existante, sans execution serveur en |
| `incident open <label> <severity> <summary>` | Service | Non | Ouvre un workspace incident local append-only. |
| `incident list [query]` | Service | Non | Liste les events incidents locaux, optionnellement filtrees. |
| `incident status <label>` | Service | Non | Affiche etat, events, notes et pins locaux d'un incident. |
| `incident note <label> <text>` | Service | Non | Ajoute une note locale ciblee `incident:<label>`. |
| `incident pin <label> <source> <text>` | Service | Non | Attache une preuve log locale a un incident. |
| `incident export <label> --file <name>` | Service | Non | Exporte le workspace incident local en Markdown. |
| `incident close <label> <resolution>` | Service | Non | Ferme localement le workspace incident. |
| `ops snapshot [label]` | Service | Non | Cree un snapshot Markdown local avec dashboard, logs, services, audit, notes |
| `ops timeline [query]` | Service | Non | Affiche une timeline locale audit/incidents/notes/pins/approvals/drafts. |
| `ops handoff [label] --file <name>` | Service | Non | Exporte un rapport Markdown local de releve de shift. |
| `ops doctor [--verbose]` | Service | Non | Verifie localement l'etat du cockpit, outils, env gates, stores, docs et PD |
| `ops paths` | Service | Non | Liste les chemins locaux Core Access utiles pour audit, notes, incidents, exports et doc |
| `ops exports [query]` | Service | Non | Liste les exports locaux Core Access par date, taille et fichier. |
| `user search <query> --execute` | Service | Non | Recherche utilisateur via Nexus Identity read-only; dry-run par defa |
| `user open <user> --execute` | Service | Non | Ouvre fiche utilisateur via Nexus Identity read-only; ajoute une trace |
| `user create <user> <email> <role> --password-env ENV --confirm user:<user>:create --execute` | Admin | Oui | Cree un |
| `user edit <user> key=value --confirm user:<user>:edit --execute` | Admin | Oui | Modifie un utilisateur via Nexus Ide |
| `user disable <user> --confirm user:<user>:disable --execute` | Admin | Oui | Desactive un utilisateur et revoke sessi |

```

| `user sessions <user> --execute` | Service | Non | Liste sessions via Nexus Identity read-only; ajoute une trace audit s
| `user devices <user> --execute` | Service | Non | Liste devices via Nexus Identity read-only; ajoute une trace audit s
| `user audit <user> --execute` | Service | Non | Liste audit utilisateur via Nexus Identity read-only; ajoute une trace
| `user revoke-sessions <user> --confirm user:<user>:revoke-sessions --execute` | Admin | Oui | Revoque toutes les sessi
| `user force-password-reset <user> --confirm user:<user>:force-password-reset --execute` | Admin | Oui | Declenche un r
`user configs <user_id> [game_key] [--execute]`	Service	Non	Liste configs utilisateur via seed-config API.
`user config open <user_id> <config_id> [--execute]`	Service	Non	Ouvre une config via liste filter.
`user config diff <user_id> <config_id> <version>`	Service	Non	Draft diff version.
`user config export <user_id> <config_id> --format yaml [--execute]`	Service	Non	Exporte YAML.
`user config edit <user_id> <config_id> key=value [key=value...] --confirm user-config:<user_id>:<config_id>:edit`	A		
`lobby list [limit] [query] [visibility] [status] [offset] [--execute]`	Service	Non	Liste les lobbies via Nexus
`lobby open <lobby_id> [--execute]`	Service	Non	Ouvre un detail lobby via Nexus lobby-admin.
`lobby create <lobby_id> <name> [visibility] [owner] [description] --confirm lobby:<lobby_id>:create`	Admin	Oui	
`lobby edit <lobby_id> key=value [key=value...] --confirm lobby:<lobby_id>:edit`	Admin	Oui	Modifie lobby via Nex
`lobby close <lobby_id> --confirm lobby:<lobby_id>:close`	Service/Admin	Oui/Approval	Ferme lobby selon policy vi
`lobby lock <lobby_id> [reason] --confirm lobby:<lobby_id>:lock`	Service	Oui	Draft verrouillage lobby.
`lobby unlock <lobby_id> [reason] --confirm lobby:<lobby_id>:unlock`	Service	Oui	Draft deverrouillage lobby.
`lobby chat <lobby_id> [--execute]`	Service	Non	Filtre logs chat-api.
`lobby join-secret <lobby_id> [reason] --confirm lobby:<lobby_id>:join-secret`	Admin	Oui	Draft observer admin in
`lobby broadcast <lobby_id> <message> --confirm lobby:<lobby_id>:broadcast`	Service	Oui	Draft broadcast lobby.
`room list [limit] [query] [room_type] [connection_state] [offset] [--execute]`	Service	Non	Liste rooms via Nexu
`room open <room_id> [--execute]`	Service	Non	Ouvre snapshot room.
`room summary <room_id> [--execute]`	Service	Non	Diagnostics room.
`room events <room_id> [limit] [event_type] [severity] [offset] [source] [--execute]`	Service	Non	Evenements roo
`room logs <room_id> --follow [--execute]`	Service	Non	Logs room/AP.
`room snapshot <room_id> [--execute]`	Service	Non	Snapshot room.
`room sync <room_id> [reason] --confirm room:<room_id>:sync`	Admin	Oui	Draft sync payload.
`room pending-items <room_id> [--execute]`	Service	Non	Inspecte snapshot pour items en attente.
`room client-reports <room_id> [limit] [report_type] [severity] [source] [offset] [--execute]`	Service	Non	Rappo
`room request-skmlmi-reconnect <room_id> <player> [reason] --confirm room:<room_id>:request-skmlmi-reconnect`	Service		
`room disconnect-runtime <room_id> <player> [runtime] [reason] --confirm room:<room_id>:disconnect-runtime`	Admin	Ou	
`room give-item <room_id> <target> <item> [reason] --confirm room:<room_id>:give-item`	Admin	Oui	Draft injection
`client broadcast <user	all> <message> --confirm client:broadcast:<target>`	Service	Oui
`client maintenance-banner <user	all> <message> --confirm client:maintenance-banner:<target>`	Admin	Oui
`client force-update-prompt <user	all> <version	message> --confirm client:force-update-prompt:<target>`	Admin
`client request-relogin <user	all> <reason> --confirm client:request-relogin:<target>`	Service	Oui
`client refresh-lobby <user	all> [lobby]`	Service	Non
`client refresh-room <user	all> [room]`	Service	Non
`client request-skmlmi-reconnect <user	all> <reason> --confirm client:request-skmlmi-reconnect:<target>`	Service	Oui
`client restart-runtime <user	all> <reason> --confirm client:restart-runtime:<target>`	Admin	Oui
`client clear-cache-request <user	all> <reason> --confirm client:clear-cache-request:<target>`	Service	Approval
`client diagnostics-request <user> <incident> <reason> --include <set>`	Service	Consentement	Prepare une demande
`client diagnostics-list [query]`	Service	Non	Liste les demandes locales de diagnostics client.
`client diagnostics-export [query] --file <name>`	Service	Non	Exporte les demandes diagnostics et le contrat de
`broadcast global <message> --confirm broadcast:global:all`	Admin	Oui	Cree un draft audite global; aucun broadca
`broadcast server <server> <message> --confirm broadcast:server:<server>`	Admin	Oui	Cree un draft audite cible s
`broadcast lobby <lobby> <message> --confirm broadcast:lobby:<lobby>`	Service	Oui	Cree un draft audite lobby; au
`broadcast room <room> <message> --confirm broadcast:room:<room>`	Service	Oui	Cree un draft audite room; aucun b
`broadcast role <role> <message> --confirm broadcast:role:<role>`	Admin	Oui	Cree un draft audite role; aucun bro
`broadcast version <version> <message> --confirm broadcast:version:<version>`	Admin	Oui	Cree un draft audite ver
`broadcast game <game> <message> --confirm broadcast:game:<game>`	Service	Oui	Cree un draft audite jeu; aucun br
`maintenance status`	Service	Non	Etat maintenance.
`maintenance enable <scope> <message> --confirm maintenance:<scope>:enable`	Admin	Oui	Cree un draft audite d'act
`maintenance disable <scope> [reason] --confirm maintenance:<scope>:disable`	Admin	Oui	Cree un draft audite de d
`maintenance schedule <scope> <start> <end>`	Admin	Oui	Planifie maintenance.
`maintenance broadcast <scope> <message> --confirm maintenance:<scope>:broadcast`	Admin	Oui	Cree un draft audite
`maintenance history`	Service	Non	Historique.
`release current`	Service	Non	Affiche le dernier manifeste local client update-bundle et ses URLs/fallbacks.
`release list`	Service	Non	Liste les manifestes locaux dedupliques par date/version/channel/build.
`release verify [latest	version	date	manifest] [--fast]`
`release verify-cdn [channel] [platform	all] --execute`	Service	Non
`release compare <a> <b>`	Service	Non	Compare deux manifestes locaux par version, URL, artefacts, tailles et has
`release publish <manifest	version	date> --confirm release:<version>:publish`	Admin

```

```

| `release rollback <version> --confirm release:<version>:rollback` | Admin | Oui | Cree un draft audite de rollback; ne
| `release schedule <manifest|version|date> <datetime> --confirm release:<version>:schedule` | Admin | Oui | Cree un dra
`release notes [version	date	manifest]`	Service	Non	Affiche le manifeste local et les drafts release lies.
`release audit [version	date	manifest]`	Service	Non	Affiche le manifeste local et les drafts release lies.
`client-banner list`	Service	Non	Liste 3 slots.		
`client-banner edit <slot>`	Admin	Oui	Edite slot.		
`client-banner preview <slot>`	Service	Non	Preview.		
`client-banner publish <slot> --confirm banner:<slot>:publish`	Admin	Oui	Cree un draft audite de publication ban		
`client-banner rollback <slot> --confirm banner:<slot>:rollback`	Admin	Oui	Cree un draft audite rollback banner;		
`client-banner disable <slot> --confirm banner:<slot>:disable`	Admin	Oui	Cree un draft audite disable banner; au		
`pack repo list`	Service	Non	Liste repos packs.		
`pack repo add <id> <url> <game> [notes] --confirm pack-repo:<id>:add`	Admin	Oui	Cree un draft repo pack; aucun		
`pack repo edit <id> key=value --confirm pack-repo:<id>:edit`	Admin	Oui	Cree un draft edition repo pack; aucun p		
`pack repo disable <id> [reason] --confirm pack-repo:<id>:disable`	Admin	Oui	Cree un draft disable repo pack; au		
`pack repo delete <id> [reason] --confirm pack-repo:<id>:delete`	Admin	Oui	Cree un draft delete repo pack; aucun		
`pack check <id> [notes]`	Service	Non	Cree un draft check pack; aucun fetch reseau.		
`pack validate <id> [notes]`	Service	Non	Cree un draft validate pack; aucune logique Lua executee.		
`pack stage <id> [notes] --confirm pack:<id>:stage`	Admin	Oui	Cree un draft stage pack; aucun artifact CDN stage		
`pack publish <id> [notes] --confirm pack:<id>:publish`	Admin	Oui	Cree un draft publish pack; aucun artifact CDN		
`pack rollback <id> <version> --confirm pack:<id>:rollback:<version>`	Admin	Oui	Cree un draft rollback pack; auc		
`pack schedule-check <id> <cron	interval> --confirm pack:<id>:schedule-check`	Admin	Oui	Cree un draft scheduler	
`schedule list`	Service	Non	Liste jobs.		
`schedule calendar`	Service	Non	Vue calendrier.		
`schedule add <name> <when> <command> --confirm schedule:<name>:add`	Admin	Oui	Ajoute un draft job local; job no		
`schedule edit <job> key=value --confirm schedule:<job>:edit`	Admin	Oui	Cree un draft edition scheduler; aucun j		
`schedule pause <job> [reason] --confirm schedule:<job>:pause`	Admin	Oui	Cree un draft pause scheduler; aucun jo		
`schedule resume <job> [reason] --confirm schedule:<job>:resume`	Admin	Oui	Cree un draft resume scheduler; aucun		
`schedule run-now <job> [reason] --confirm schedule:<job>:run-now`	Admin	Oui	Cree un draft run-now; aucun job ex		
`schedule history`	Service	Non	Historique local des jobs/drafts scheduler.		
`discord status [--execute]`	Service	Non	Statut bot Discord via service allowliste.		
`discord reload --confirm discord:reload:bot`	Admin	Oui	Draft reload bot.		
`discord announce <channel> <message> --confirm discord:announce:<channel>`	Service	Oui	Draft annonce Discord.		
`discord sync-roles --confirm discord:sync-roles:all`	Admin	Oui	Draft sync roles.		
`discord command list`	Service	Non	Liste commandes.		
`discord command enable <name> [detail] --confirm discord:command:<name>:enable`	Admin	Oui	Draft activation comm		
`discord command disable <name> [detail] --confirm discord:command:<name>:disable`	Admin	Oui	Draft desactivation		
`discord command edit <name> [detail] --confirm discord:command:<name>:edit`	Admin	Oui	Draft edition commande.		
`discord timer list`	Service	Non	Liste timers.		
`discord timer edit <id> key=value [key=value...] --confirm discord:timer:<id>:edit`	Admin	Oui	Draft edition tim		
`discord incident-post <incident> [channel] [message] --confirm discord:incident-post:<incident>`	Service	Oui	Dr		
`discord logs [--follow] [--execute]`	Service	Non	Logs Discord.		
`twitch status [--execute]`	Service	Non	Statut bot Twitch via service allowliste.		
`twitch connect <channel> [reason] --confirm twitch:connect:<channel>`	Admin	Oui	Draft connexion channel.		
`twitch disconnect <channel> [reason] --confirm twitch:disconnect:<channel>`	Admin	Oui	Draft deconnexion channel		
`twitch announce <channel> <message> --confirm twitch:announce:<channel>`	Service	Oui	Draft annonce Twitch.		
`twitch command list`	Service	Non	Liste commandes.		
`twitch command enable <name> [detail] --confirm twitch:command:<name>:enable`	Admin	Oui	Draft activation comman		
`twitch command disable <name> [detail] --confirm twitch:command:<name>:disable`	Admin	Oui	Draft desactivation c		
`twitch command edit <name> [detail] --confirm twitch:command:<name>:edit`	Admin	Oui	Draft edition commande.		
`twitch timer list`	Service	Non	Liste timers.		
`twitch timer edit <id> key=value [key=value...] --confirm twitch:timer:<id>:edit`	Admin	Oui	Draft edition timer		
`twitch lobby announce <lobby> [message] --confirm twitch:lobby:<lobby>:announce`	Service	Oui	Draft annonce lobb		
`twitch stream set-title-hint [channel] <title>`	Service	Non	Draft aide titre stream.		
`twitch logs [--follow] [--execute]`	Service	Non	Logs Twitch.		
`cleanup plan logs`	Admin	Non	Cree un draft plan cleanup logs; aucun scan/suppression.		
`cleanup plan db`	Admin	Non	Cree un draft plan cleanup DB; aucune mutation DB.		
`cleanup plan spool`	Admin	Non	Cree un draft plan cleanup spool; aucune suppression.		
`cleanup plan all`	Admin	Non	Cree un draft plan cleanup global; aucun scan/suppression.		
`cleanup apply <plan_id> --confirm cleanup:<plan_id>:apply`	Admin	Oui	Cree un draft apply cleanup; aucune mutati		
`cleanup history`	Service	Non	Liste les drafts cleanup locaux.		
`cleanup rollback <id> --confirm cleanup:<id>:rollback`	Admin	Oui	Cree un draft rollback cleanup; aucune mutatio		
`audit search`	Service	Non	Recherche audit.		

```

18 16-training-moderators

16 - Formation modérateurs

Objectif

Former le rôle Service pour aider en live sans mettre la production en danger.

Parcours

1. Lire Concepts.
2. Lire Premier login.
3. Lire Navigation TUI.
4. Lire Rôles et permissions.
5. Faire les exercices logs.
6. Faire les exercices lobbies/rooms.
7. Faire un incident simulé.
8. Lire les limites Service.

Exercices

Exercice 0 - Vérifier le poste Core Access

```
```text
auth whoami
ops doctor
ops paths
ops exports
```
```

But: confirmer le rôle, l'état local du cockpit, les chemins de travail, et les exports déjà présents avant un shift.

Exercice 1 - Diagnostiquer un joueur

```
```text
user search <name>
user open <user>
user sessions <user>
user configs <user>
```
```

But: trouver si le joueur est connecté, sur quel device et avec quelle config.

Exercice 2 - Suivre une room

```
```text
room summary <room>
room events <room>
room logs <room> --follow
```
```

But: identifier qui est connecté, quel runtime parle, et s'il y a des pending items.

Exercice 3 - Créer un incident local

```
```text
incident open training-room sev4 training incident
incident note training-room player reported a reconnect issue
incident pin training-room link:room-runtime sample runtime line
incident status training-room
incident export training-room --file training-room.md
```
```

incident close training-room resolved during training
```

But: comprendre le journal append-only, les notes, les pins et l'export  
Markdown sans toucher a la production.

### Exercice 4 - Faire une releve de shift

```text  
ops timeline training-room
ops handoff training-shift --file training-shift.md
```

But: produire un rapport de releve lisible pour un autre modérateur.

## A ne jamais faire sans Admin

- publier une release;
- rollback release;
- reboot/update serveur;
- modifier role;
- modifier secret;
- injecter item;
- modifier DB critique;
- ignorer un backup gate.

## Checklist avant premier shift

- Je sais me connecter au bastion.
- Je sais verifier mon role.
- Je sais lancer `ops doctor` avant un shift.
- Je sais lister logs et rooms.
- Je sais creer un incident local, une note et un pin.
- Je sais produire une releve de shift.
- Je sais demander approval.
- Je sais utiliser F12 Panic en lecture avant action.



## 19 17-admin-runbook

### # 17 - Runbook Admin

#### ## Release day

1. ``ops doctor --verbose``
2. ``release current``
3. ``release verify-cdn``
4. ``schedule calendar``
5. ``maintenance schedule client_update_only <start> <end>`` si necessaire.
6. Publier via ``release publish <manifest>``.
7. Verifier clients Linux/Windows.
8. Creer ops note avec resultat.

#### ## Stream/live event

1. ``ops doctor``
2. Ouvrir dashboard dense.
3. ``incident open stream-<date> sev4 live monitoring``
4. Panels recommandes: Link logs, room logs, lobby list, alerts, Discord/Twitch.
5. Garder F12 Panic pret.
6. Tout incident important devient ``incident note`` ou ``incident pin``.
7. ``ops exports``
8. ``ops handoff stream-<date> --file stream-<date>.md`` en fin de stream.

#### ## Incident majeur

1. ``incident open major-<date>-<scope> sev1 <summary>``
2. ``server agent-health all``
3. ``server agent-system <server>``
4. ``server agent-services <server>``
5. Stabiliser: maintenance scope minimal.
6. Broadcast.
7. Snapshot rooms.
8. Collect logs bundle.
9. Identifier service.
10. ``server agent-logs <server> <service>``
11. Redemarrer seulement avec Admin, gate mutation, et confirmation exacte:  
``server restart <server> <service> --confirm <server>:<service>:restart --execute``
12. Appliquer rollback seulement si impact clair.
13. ``incident close major-<date>-<scope> <resolution>``
14. Clore avec ops commit et postmortem.

#### ## Audit review

1. ``audit search``
2. Filtrer par date/role/action.
3. Exporter Markdown/JSON.
4. ``ops timeline [query]``
5. ``ops paths``
6. ``ops exports [query]``
7. Verifier actions sans rollback.
8. Creer notes de follow-up.

## 20 glossary

# Glossaire

**\*\*Admin\*\***: role complet avec actions dangereuses, approvals et maintenance.

**\*\*Approval queue\*\***: file de demandes preparees par Service et approuvees par Admin.

**\*\*Bastion Nexus\*\***: serveur Nexus utilise comme point d'entree SSH et hote de Core Access.

**\*\*Client Core\*\***: app desktop SekaiLink utilisateur.

**\*\*Core Access\*\***: cockpit TUI admin. Ne pas confondre avec Client Core ou les libretro cores.

**\*\*Evolution\*\***: role distribution, CDN, assets, packs et releases.

**\*\*Link\*\***: API publique, lobbies, chat, room passthrough, release-latest.

**\*\*Nexus\*\***: identite, users, configs, audit, scheduler et source de confiance.

**\*\*Ops commit\*\***: entree atomique d'audit pour une action.

**\*\*Pulse\*\***: assistant/config helper.

**\*\*Room\*\***: session runtime/live associee a une sync.

**\*\*Sekaiemu\*\***: emulator renderer/runtime visuel.

**\*\*Service\*\***: role support/moderation puissant, avec approvals pour actions dangereuses.

**\*\*SKLMI\*\***: companion tracker/runtime qui possede la logique tracker.

**\*\*Worlds\*\***: generation service.

## 21 README

### # Change Control

Ce dossier definit la gouvernance des futures modifications liees a SekaiLink Core Access.

### ## Regles

- Toute modification code doit avoir sa documentation dans le meme commit.
- Les changements serveur/client/emu doivent documenter leur impact.
- Les changements de connectivite Client Core, Sekaiemu ou SKLMI exigent un contrat complet.
- Toute modification SKLMI exige un accord explicite ecrit de Jade.
- Les changements urgents live peuvent etre patches d'abord, mais doivent etre documentes immediatement dans la meme session.

### ## Templates

- [Change record](template-change-record.md)
- [Connectivity contract](template-connectivity-contract.md)
- [SKLMI approval request](template-sklmi-approval-request.md)
- [Emergency change policy](emergency-change-policy.md)
- [Platform sync policy](platform-sync-policy.md)
- [Changelog](changelog.md)

## 22 emergency-change-policy

### # Emergency Change Policy

Emergency live changes are allowed only when production, stream, login, release or live room operation is actively blocked.

### ## Procedure

1. Stabilize with the smallest possible action.
2. Capture command, target and reason.
3. Patch if needed.
4. Document immediately in the same session.
5. Create or update change record before final response.
6. Add rollback and verification steps.

### ## Required minimum note

- What broke:
- Why immediate action was needed:
- What changed:
- Target:
- Risk:
- Rollback:
- Verification:

### ## SKLMI exception

There is no emergency exception for editing SKLMI without explicit approval.

## 23 platform-sync-policy

### # Platform Sync Policy

Every change record must mention Linux and Windows.

### ## Required fields

- Linux impact:
- Windows impact:
- Build needed:
- Runtime bundle impact:
- Release gate:

### ## Rule

Linux/Windows synchronization is required before a public release when the change touches Client Core, Sekaiemu, runtime packaging, SKLMI launch, updater or client signals.

For this Core Access plan, synchronization is documented but not implemented.